# Towards Better Statistical Understanding of Watermarking LLMs

## Based on the 2024 Paper by Cai et al.

Nate Annau

UC Santa Barbara

2026-03-07

# Outline

1. Introduction ..... 2
2. Distortion vs Detection Ability ..... 7
3. Main Contribution ..... 13
4. Experimental Results ..... 18
5. Questions? ..... 22

# 1. Introduction

# 1.1 Previous Approaches on Watermarking Natural Language

- "Gumbel Softmax" Scheme — *Aaronson (2023); Christ et al. (2023)*
- Partition vocabulary vocabulary and distort probabilities — *Kirchenbauer et al. (2023)*
- Unique assignments of pseudorandom functions per token via secret keys — *Kuditipudi et al. (2023)*
- Curve Pareto optimum against log perplexity — *Wouters (2023)*

Key Question: *What is the minimal price for a watermarking algorithm to attain a certain level of detection ability?*

# 1.2 Model Setup

A language model (LM) $\mathcal{M}$ is a function that treats both a prompt $x$ and previous $t-1$ tokens $y_1, \ldots, y_{t-1}$ as *context* $\mathbf{y}_{[t-1]}$ and maps this to a probability vector $\mathbf{p}_t \in \Delta(\mathcal{V})$ as a prediction, where $\Delta(\mathcal{V})$ denotes the set of all probability distributions over a vocabulary $\mathcal{V}$.

The probability vector is obtained by applying softmax to a logit vector $\mathbf{l}_t = (l_{t,1}, \ldots, l_{t,\mathcal{V}})$:

$$p_{t,k} := \frac{\exp(l_{t,k})}{\sum_{k' \in \mathcal{V}} \exp(l_{t,k'})}.$$

New tokens $y_t$ are sampled until a terminate symbol $v_{\text{term}} \in \mathcal{V}$ is sampled or a maximum length $T_{\max}$ is reached.

# 1.3 Defining Watermarking

We wish to develop a model $\mathcal{M}'$ based on a LM $\mathcal{M}$ together with a *detection algorithm*, fitting the following ideal properties:

- The change in output is small enough so as to be unnoticeable to a human
- The change in output is large enough so as to be detectable by the algorithm with high certainty

We want to formalize this tradeoff, clearly defining the classes of error at play.

# 1.4 Classes of Error

Let $\mathcal{K}$ be the space of all possible detection keys $K$, and $\mathcal{T}$ be the space of token sequences $\mathbf{y}_{[t]} = y_1, \ldots, y_t$.

Define a **watermarked model** $\mathcal{L}'$ as consisting of an algorithm & key pair $(\mathcal{A}, \mathcal{K})$ where $\mathcal{A} : K \times \mathcal{T} \to \Delta(\mathcal{V})$, i.e., we get a PMF over $\mathcal{V}$.

A **detection procedure** $\mathcal{D} : K \times \mathcal{T} \to \{0, 1\}$ then determines whether the text came from $\mathcal{L}'$.

Then the two types of errors are (for some prompt $\mathbf{x} = \mathbf{y}_{[n<t]}$):

$$\alpha_{\mathbf{p}}(\mathcal{D}, \mathbf{x}) := \mathbb{P}\Big(\mathcal{D}\Big(K, \mathbf{y}_{[T_{\max}]}\Big) = 1 \mid \mathbf{y}_{[T_{\max}]} \sim \mathbf{p}(x)\Big) \qquad \text{(false positives)}$$

$$\beta(\mathcal{D}, \mathbf{x}) := \mathbb{P}\Big(\mathcal{D}\Big(K, \mathbf{y}_{[T_{\max}]}\Big) = 0 \mid \mathbf{y}_{[T_{\max}]} \sim \mathbf{q}^{(\mathcal{A},K)}(x)\Big) \qquad \text{(false negatives)}$$

# 2. Distortion vs Detection Ability

# 2.1 Kirchenbauer et al.

GENERALIZED SOFT WATERMARKING

$t \leftarrow 1, y_0 \leftarrow k$
**while** $t \leq T_{\max}$ and $y_{t-1} \neq v_{\text{term}}$ **do**
  Use $f(k, y_{t-1})$ to randomly partition $\mathcal{V}$ into a *green list* $G$ with $|G| = \gamma|\mathcal{V}|$ and a *red list* $|G^c| = (1-\gamma)|\mathcal{V}|$
  $\mathbf{q}_t \leftarrow \text{softmax}(\mathbf{p}_t + \boldsymbol{\delta}_t \cdot \mathbb{1}_G)$
  Sample a token $y_t \sim \left(q_{t,1}, \ldots, q_{t,|\mathcal{V}|}\right)$
  $t \leftarrow t + 1$
**end while**
**if** $t < T_{\max}$ **then**
  Set all remaining $y_{t+1}, \ldots, y_{T_{\max}}$ to be $v_{\text{term}}$
**end if**

**Input**

1. Language Model $\{\mathbf{p}\}_{t \in [T_{\max}]} \in \Delta(\mathcal{V})$
2. Prompt $\mathbf{x} \in \mathcal{T}$
3. Green list ratio $\gamma \in (0, 1)$
4. Pseudorandom function $f$
5. Perturbations $\{\boldsymbol{\delta}\}_{t \in [T_{\max}]} \in \mathbb{R}_{>0}^{|\mathcal{V}|}$
6. Random seed $k$

**Output**

1. Watermarked token sequence $\mathbf{y}_{[T_{\max}]}$

# 2.2 Distance Measures

With this manipulation, the idea of our detection algorithm $\mathcal{D}$ is simple: how many more green words were there than expected?

**Definition: Difference of Green Word Probability (DG)**

$$\mathrm{DG}_t(\mathbf{q}_t) := \sum_{v \in G} q_{t,v} - \sum_{v \in G} p_{t,v}.$$

This gives us a distinguishability condition. Further, KL divergence gives us a natural distance measure between our PMFs:

$$D_{\mathbf{p}}(\mathbf{q}_t) := D(\mathbf{q}_t, \mathbf{p}_t).$$

# 2.3 Constrained Optimization Formulation

Thus, we frame our goal as the following constrained optimization problem:

$$\mathrm{OPT}(\Delta) := \min_{\boldsymbol{\delta}_t} \frac{1}{T} \sum_{t=1}^{T} D_{\mathbf{p}}(\mathbf{q}_t(\boldsymbol{\delta}_t))$$

$$\text{such that } \frac{1}{T} \sum_{t=1}^{T} \mathrm{DG}_t(\mathbf{q}_t(\boldsymbol{\delta}_t)) \geq \Delta.$$

where the PMFs $\mathbf{q}_t$ of the watermarked LM are treated as functions of $\boldsymbol{\delta}_t$ alone for brevity.

# 2.4 Special Cases

We can make our problem

$$\mathrm{OPT}(\Delta) := \min_{\boldsymbol{\delta}_t} \frac{1}{T} \sum_{t=1}^{T} D_{\mathbf{p}}(\mathbf{q}_t(\boldsymbol{\delta}_t))$$

$$\text{such that } \frac{1}{T} \sum_{t=1}^{T} \mathrm{DG}_t(\mathbf{q}_t(\boldsymbol{\delta}_t)) \geq \Delta.$$

simpler by

- restricting $\delta_{t,1} = \cdots = \delta_{t,|\mathcal{V}|} = \delta_t$ $\qquad \mathrm{OPT}_{\text{static } v}(\Delta)$
- restricting $\delta_{t,k} = \delta \forall t \in [T_{\max}], v \in \mathcal{V}$ $\qquad \mathrm{OPT}_{\text{static } t,v}(\Delta)$

# 2.5 Generalization Weakness

We can deduce that the objective function can be viewed as an unbiased estimator for the KL divergence between the LMs.

***Proposition***

$$\mathbb{E}\left[\sum_{t=1}^{T_{\max}} D_{\mathbf{p}}(\mathbf{q}_t(\boldsymbol{\delta}_t))\right] = \sum_{t=1}^{T_{\max}} D\Big(\mathbf{q}_{t|[t-1]}(\mathbf{x}) \| \mathbf{p}_{t|[t-1]} | \mathbf{q}_{[t-1]}(x)\Big)$$
$$= D(\mathbf{q}(\mathbf{x}) \| \mathbf{p}(\mathbf{x}))$$

This leads to two important results.

***Proposition***

Suppose $\mathrm{OPT}(\Delta)$ is feasible. Then its optimal solution shares the same form of $\mathrm{OPT}_{\text{static } v}(\Delta)$.

***Proposition***

Suppose $\mathrm{OPT}_{\text{static } v}(\Delta)$ is feasible. Then its optimal solution shares the same form of $\mathrm{OPT}_{\text{static } t,v}(\Delta)$.

So, generalizing $\delta$ to $\boldsymbol{\delta}_t$ brings no additional benefit. Further, it is forcibly online and, when viewing this as a multi-objective optimization problem, does not result in the Pareto optimum.

# 3. Main Contribution

# 3.1 An Online Algorithm with Adaptive $\delta$



We desire an algorithm that

- Ensures Pareto optimality
- Achieves the detection threshold $\Delta$
- Converges to optimal $\delta^*$ quickly

Our idea is to perform an online gradient ascent on the Lagrangian dual function for the dual variable $\lambda_t$, and use this to approximate the optimal dual variable $\lambda^*$. To see this, first introduce the Lagrangian of $\mathrm{OPT}_{\text{static } t,v}(\Delta)$:

$$\begin{aligned} L(\delta, \lambda) &:= \frac{1}{T}\sum_{t=1}^{T} D_{\mathbf{p}}(\mathbf{q}_t(\delta)) - \lambda \cdot \left( \frac{1}{T}\sum_{t=1}^{T} \mathrm{DG}_t(\delta) - \Delta \right) \\ &= \frac{1}{T}\sum_{t=1}^{T} D_{\mathbf{p}}(\mathbf{q}_t(\delta)) - \frac{1}{T}\sum_{t=1}^{T} \lambda(\mathrm{DG}_t(\delta) - \Delta). \end{aligned}$$

Denote the corresponding primal function and dual functions by

$$f(\delta) := \sup_{\lambda \geq 0} L(\delta, \lambda) \qquad\qquad g(\lambda) := \inf_{\delta} L(\delta, \lambda).$$

# 3.2 Key Properties of the Optimization Problem

***Lemma***

1. The infimum defining $g(\lambda)$ can always be achieved by setting $\delta = \lambda$.
2. The Lagrangian dual $g(\lambda)$ can be decomposed token-wise by defining $L_t(\delta) := D_{\mathbf{p}}(\delta) - \lambda(\mathrm{DG}_t(\delta) - \Delta)$. Then with $f_t$ and $g_t$ defined accordingly, part 1 holds for each $g_t(\lambda)$, and

$$g(\lambda) = \frac{1}{T}\sum_{t=1}^{T} g_t(\lambda)$$

   where each $g_t$ is concave with $\frac{\mathrm{d}g_t}{\mathrm{d}\lambda} = \Delta - \mathrm{DG}_t(\lambda)$.
3. If the primal problem is feasible, then the strong duality holds:

$$\inf_{\delta} f(\delta) = \sup_{\lambda \geq 0} g(\lambda).$$

   with $\delta^* = \lambda^*$, where $\lambda^*$ is exactly the optimal $\lambda$ for maximizing the Lagrangian dual $g(\lambda)$.

# 3.3 Dual Gradient Ascent Algorithm



Dual Gradient Ascent for Soft Watermarking (henceforth *DualGA*)

$t \leftarrow 1, y_0 \leftarrow k$

**while** $t \leq T_{\max}$ and $y_{t-1} \neq v_{\text{term}}$ **do**

  Use $f(k, y_{t-1})$ to randomly partition $\mathcal{V}$ into a *green list* $G$ with $|G| = \gamma|\mathcal{V}|$ and a *red list* $|G^c| = (1-\gamma)|\mathcal{V}|$

  Set $\delta_t$ according to the Lagrangian dual

$$\delta_t \leftarrow \lambda_t$$

  $\mathbf{q}_t \leftarrow \text{softmax}(\mathbf{p}_t + \delta_t \cdot \mathbb{1}_G)$

  Sample a token $y_t \sim \left(q_{t,1}, \ldots, q_{t,|\mathcal{V}|}\right)$

  Compute the online gradient of the dual function

$$\text{gd}_t \leftarrow \Delta - \text{DG}_t(\delta_t)$$

  Update the dual variable via online gradient ascent

$$\lambda_{t+1} \leftarrow \lambda_t + \eta \cdot \text{gd}_t$$

  $t \leftarrow t + 1$

**end while**

**if** $t < T_{\max}$ **then**

  Set all remaining $y_{t+1}, \ldots, y_{T_{\max}}$ to be $v_{\text{term}}$

**end if**

**Input**

1. Language Model $\{\mathbf{p}\}_{t \in [T_{\max}]} \in \Delta(\mathcal{V})$
2. Prompt $\mathbf{x} \in \mathcal{T}$
3. Green list ratio $\gamma \in (0, 1)$
4. Pseudorandom function $f_\gamma$
5. DG constraint $\Delta$
6. Step size $\eta$
7. Initial dual variable $\delta_1$
8. Random seed $k$

**Output**

1. Watermarked token sequence $\mathbf{y}_{[T_{\max}]}$

# 3.4 Algorithm Analysis



We assume that

1. Green word probabilities are i.i.d. random variables for all $t \in [T]$
2. The optimal dual variable $\lambda^*$ is bounded: $\lambda^* \in [0, M]$, and $\lambda$ starts and ends in this range.

Then with $\eta = \Theta\left(\frac{1}{\sqrt{T}}\right)$, we have that:

***Theorem***

The previous algorithm satisfies

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[D_{\mathbf{p}}(\mathbf{q}_t(\delta_t))\right] \leq \mathbb{E}[\mathrm{OPT}(\Delta)] + O\left(\frac{1}{\sqrt{T}}\right)$$

and

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[\mathrm{DG}_{\mathbf{p}}(\mathbf{q}_t(\delta_t))\right] \geq \Delta - O\left(\frac{1}{\sqrt{T}}\right).$$

# 4. Experimental Results

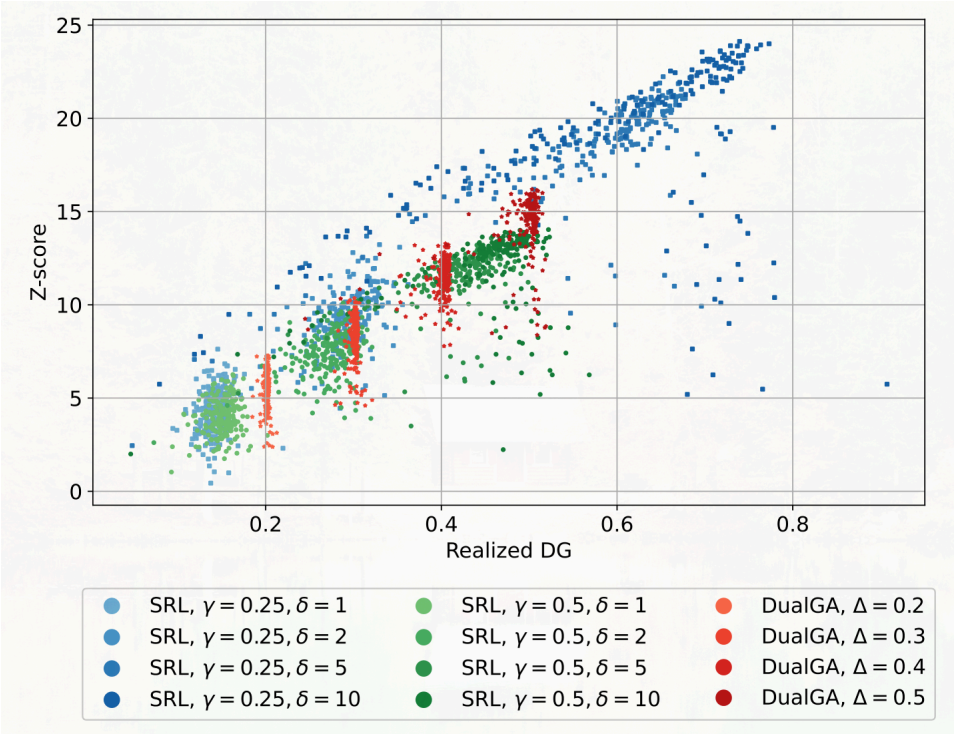
# 4.1 Kirchenbauer Implementation

```
for _ in range(max_new_tokens):
    outputs = model(next_input, past_key_values=past_key_values,
use_cache=True)
    past_key_values = outputs.past_key_values
    logits = outputs.logits[:, -1, :]  # (1, vocab_size)

    # 3. Get green list based on previous token
    prev_token = int(generated[0, -1:].item())
    green_set = get_green_list(prev_token, vocab_size, gamma)
    green_mask = torch.zeros(vocab_size, device=device)
    green_indices = torch.tensor(list(green_set), device=device,
dtype=torch.long)
    green_mask[green_indices] = 1.0

    # p_t: original probabilities
    original_probs = torch.softmax(logits, dim=-1).squeeze(0)

    # Add delta to green list logits
    watermarked_logits = logits.clone()
    watermarked_logits[0, green_indices] += delta

    # 4. Watermarked probabilities
    wm_probs = torch.softmax(watermarked_logits, dim=-1).squeeze(0)

    # Compute DG_t = sum_{v in G} q_{t,v} - sum_{v in G} p_{t,v}
    green_prob_original = (original_probs * green_mask).sum().item()
    green_prob_wm = (wm_probs * green_mask).sum().item()
    dg_t = green_prob_wm - green_prob_original
    dg_values.append(dg_t)

    # Compute KL(q_t || p_t)
    kl_t = (wm_probs * (torch.log(wm_probs + 1e-30) -
torch.log(original_probs + 1e-30))).sum().item()
    kl_values.append(kl_t)

    # 5. Sample from watermarked distribution
    next_token = torch.multinomial(wm_probs.unsqueeze(0),
num_samples=1)
    generated = torch.cat([generated, next_token], dim=-1)
    next_input = next_token

    # Track if sampled token is green
    is_green = next_token.item() in green_set
    green_hits.append(1.0 if is_green else 0.0)

    # Stop on EOS
    if next_token.item() == tokenizer.eos_token_id:
        break
```

# 4.3 Dual GA Implementation

```python
for t in range(max_new_tokens):
    outputs = model(next_input, past_key_values=past_key_values, use_cache=True)
    past_key_values = outputs.past_key_values
    logits = outputs.logits[:, -1, :]  # (1, vocab_size)

    # 3. Get green list based on previous token
    prev_token = int(generated[0, -1:].item())
    green_set = get_green_list(prev_token, vocab_size, gamma)
    green_mask = torch.zeros(vocab_size, device=device)
    green_indices = torch.tensor(list(green_set), device=device, dtype=torch.long)
    green_mask[green_indices] = 1.0

    # p_t: original probabilities
    original_probs = torch.softmax(logits, dim=-1).squeeze(0)

    # g = sum of original probs on green list
    g = (original_probs * green_mask).sum()

    # 4. delta = lambda
    delta_t = max(0.0, min(lambda_t, 15.0))

    # Add delta to green list logits
    watermarked_logits = logits.clone()
    watermarked_logits[0, green_indices] += delta_t

    # 5. Watermarked probabilities
    wm_probs = torch.softmax(watermarked_logits, dim=-1).squeeze(0)

    # Compute DG_t
    green_prob_original = (original_probs * green_mask).sum().item()
    green_prob_wm = (wm_probs * green_mask).sum().item()
    dg_t_val = green_prob_wm - green_prob_original
    dg_values.append(dg_t_val)

    # Compute KL(q_t || p_t)
    kl_t = (wm_probs * (torch.log(wm_probs + 1e-30) - torch.log(original_probs +
    1e-30))).sum().item()
    kl_values.append(kl_t)

    # 6. Sample from watermarked distribution
    next_token = torch.multinomial(wm_probs.unsqueeze(0), num_samples=1)
    generated = torch.cat([generated, next_token], dim=-1)
    next_input = next_token

    # Track if sampled token is green
    is_green = next_token.item() in green_set
    green_hits.append(1.0 if is_green else 0.0)

    # DG_t as a function of delta and g.
    # Not sure if this is explicitly given in the paper but it's in their code
    g_val = g.item()
    delta_tensor = torch.tensor(delta_t)
    g_tensor = torch.tensor(g_val)
    denom = g_tensor * torch.exp(delta_tensor) - g_tensor + 1
    dg_closed = (1 - g_tensor) * (1 - 1 / denom)

    # Gradient: d g_t / d lambda = Delta - DG_t(lambda)
    gradient = D_target - dg_closed.item()

    # Step size: eta = autoeta / sqrt(t+1)
    eta = autoeta / np.sqrt(t + 1)

    # Update lambda (projected gradient ascent)
    lambda_t = max(0.0, min(lambda_t + eta * gradient, 15.0))

    # Stop on EOS
    if next_token.item() == tokenizer.eos_token_id:
        break
```

# 4.4 Comparison of Results



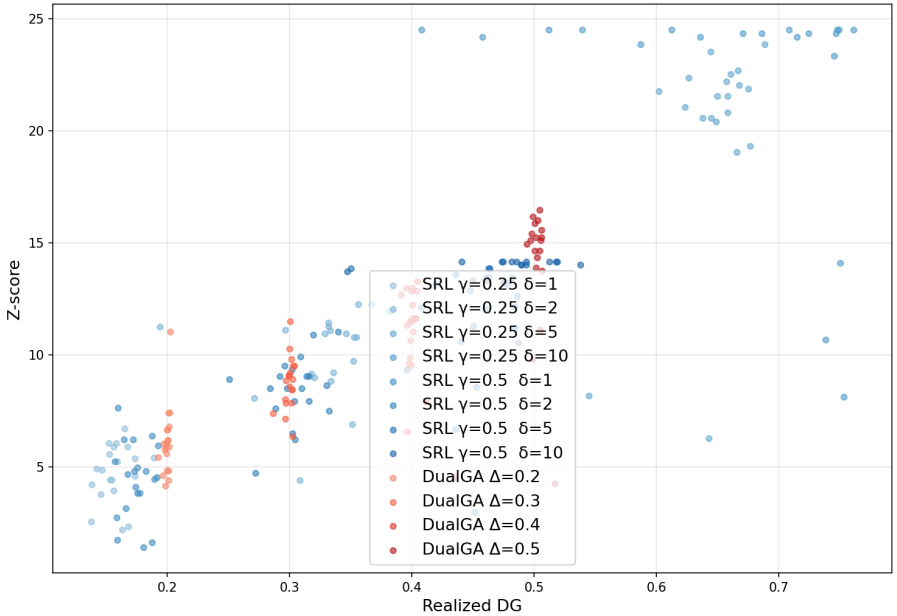




Figure 1: The scatter plot of $z$-score vs realized DG for different algorithms. SRL stands for the Kirchenbauer algorithm, and DualGA is the Dual Gradient Ascent algorithm. Each point represents one generated sequence, and for each algorithm 200 sequences are generated.

# 5. Questions?